

A Crash Course in Arduino Programming

Lesson 4

A. Introduction

Welcome to the fourth and final lesson of the Arduino Programming Crash Course. Today, we are learning all about words! We'll be covering characters, strings, Strings(), and Serial.



B. Characters

Characters are variables holding a singular symbol. Each symbol has a corresponding numerical value determined by its **character encoding**. In Arduino, we use the **American Standard Code for Information Exchange (ASCII)**¹.

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

¹ If you organize characters in order of their ASCII code, it's called ASCIIbetical order.

We can refer to characters using either the character itself or its numeric code. We can perform operations on characters like numeric values. For example 'A' is the same as 65 and ('@'+1). 'c' is equivalent to ('!'*3). I verified these statements with the following code:

```
void setup() {
  Serial.begin(9600);
  while (!Serial){}

  Serial.println(('A'==65) && ('A'==('@'+ 1)));
  Serial.println(('c'==('!'*3)));
}
```

I encourage you to pick some characters from the ASCII table and try this out yourself! The syntax for creating a character is as follows:

```
char <variable name> = <value>;
```

As mentioned previously, you can use the actual symbol, or the numeric code as the value.

```
void setup() {
  Serial.begin(9600);
  while (!Serial){}

  char char1 = 'A';
  char char2 = 65;
  char char3 = 0x41;

  if ((char1 == char2) && (char1 == char3)) {
    Serial.println("True");
  } else { Serial.println("False"); }
}
```

The code above will print True to the Serial Monitor². Notice how the words in our `Serial.println()` calls are in double quotes (") while our character symbols are in single quotes ('). Double quotes are used for strings, and single quotes are used for characters.

² What happens if you use `(char1 == char2 == char3)` as your condition?

C. string

A **string** is an array of characters. All of the following are valid ways of creating a string³.

```
void setup() {
  Serial.begin(9600);
  while (!Serial){}

  char myString[] = {'H', 'E', 'L', 'L', 'O'};
  Serial.println(myString);

  char myString2[] = "HELLO AGAIN";
  Serial.println(myString2);

  char myString3[] = "HELLO "
                    "HELLO "
                    "AGAIN "
                    "AGAIN";
  Serial.println(myString3);

  char *myString4[] = {"Is this like the arrays you've seen before?\n",
                    "It's an array of arrays!!!\n", "Fascinating!!"};
  for (int ii = 0; ii<3; ii++) {
    Serial.print(myString4[ii]);
  }
}
```

```
HELLO
HELLO AGAIN
HELLO HELLO AGAIN AGAIN
Is this like the arrays you've seen before?
It's an array of arrays!!!
Fascinating!!
```

Take a look at myString4. You'll notice the * right before the variable name. This indicates this is actually an array of **pointers**. You have to use pointers in order to make an array of arrays. Pointers *point* to where a variable is in memory.

³ This is far from an all inclusive list! Try experimenting with the array syntax you learned in lesson 3.

D. String()

Intuitively, `string` is not the same as **`String()`**⁴. “Capital-S Strings” use the `String` **class**. So, in this case you are actually making a `String` **object**, not a variable. When you create a `String`, the syntax looks less like making a variable, and more like calling a function. This is because *you are calling a function*. Objects are created from a blueprint called a **class**⁵, and you create these objects with a function called a **constructor**.

There are three ways to create a `String`:

1. `String <stringName> = String(<value>);`
2. `String <stringName> = String(<value>, <decimal places>);`
3. `String <stringName> = String(<value>, <base>);`

Option 2 can only be used when the value is a float or double. Option 3 can only be used with integers.

When it comes to choosing between using `string` or `String`, here is my advice: do not use the `String` object. It is easy and convenient when compared to `string`, but the `String` object can cause memory issues that will come back to bite you⁶.

E. Serial

When I say “Serial” there are probably three functions that come to mind: `Serial.begin()`, `Serial.print()`, and `Serial.println()`. These three functions are probably some of the first functions you were ever taught. Let’s dig into a few important Serial functions.

begin and end

`Serial.begin()` is used to set the speed at which data is transmitted in bits per second. This is called the **baud rate**. It is important that the device transmitting and the device receiving agree on the baud rate. You can demonstrate this by calling `Serial.begin()` with one baud rate and setting the Serial Monitor on your computer to a different baud rate. Chances are, whatever you try to print to the Serial Monitor will come out as gibberish. You can disable serial communication with `Serial.end()`.

print and write

`Serial.print()` prints data to the serial port as human-readable ASCII. This means numeric values passed to `Serial.print()` are converted to the values that represent the characters in that number before printing. So, `Serial.print(1)` would print “1” which is represented by decimal 49 in ASCII. To print a value without this conversion, you need to use `Serial.write()`. To print the character ‘1’ with `Serial.write()`, you could pass the value 49 (`Serial.write(49)`).

In other words: `Serial.print(49)` prints “49”. `Serial.write(49)` prints ‘1’.

⁴ Not confusing at all...

⁵ These are important concepts in object-oriented programming. We don’t cover these ideas in depth because this class is taught with an emphasis on functional programming.

⁶ Or should I say, “come back to bite you”?

read

`Serial.read()` reads incoming serial data. We need to use this function in order to collect user input from the Serial Monitor.

Serial as a condition

The keyword `Serial` can be used in if statements and while loops as the condition. It evaluates to true if the serial connection to the computer is open.

available

`Serial.available()` checks if there is data stored in the Serial **buffer** (this is where serial data goes once it has been received). `Serial.available()` is often used as a condition in a while loop so the program can keep pulling data from the buffer until the buffer is empty.